

Centro Universitário Filadélfia
Polo Salinópolis
Ciência de Dados e Inteligência Artificial

Aula 8

Testando o seu código

Fundamentos de Propriedade Intelectual Para a Transferência de
Tecnologia de Software

Thiago Rafael da Silva Moura
trsmoura@yahoo.com.br

28 de novembro de 2025

CAPÍTULO 8

Testando seu Código

Pytest, Testes Unitários e Validação de Dados
Engenharia de Software para Ciência de Dados

Roteiro da Aula

1. Fundamentos

Por que testar? Pirâmide de Testes e Tipos de Testes.

2. Pytest

Escrevendo testes, Fixtures, Parametrização e Cobertura.

3. Dados & Automação

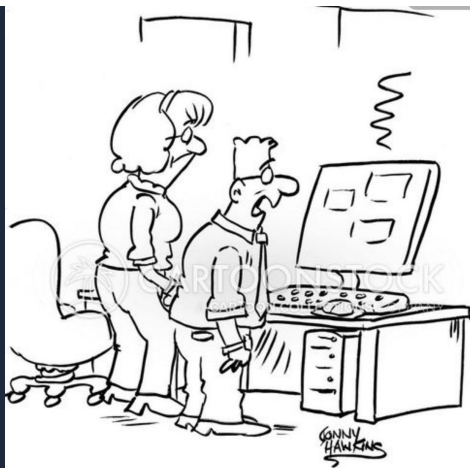
Validando dados com Pandera e Integração Contínua (CI).

O Medo de Mudar Código

Sem testes, qualquer alteração no código gera medo.
"Será que quebrei algo que funcionava?"

Isso leva ao problema clássico: *"Funciona na minha máquina"*.

Testes automatizados nos dão **confiança** para refatorar e melhorar o sistema.



"Try to think of the bugs as features."

Manual vs. Automatizado

Teste Manual

- ✓ Lento e repetitivo.
- ✓ Propenso a erro humano.
- ✓ Não documentado (está na cabeça do dev).
- ✓ Difícil de escalar.

Teste Automatizado

- ✓ Execução instantânea.
- ✓ Reprodutível e determinístico.
- ✓ Serve como documentação viva.
- ✓ Roda a cada commit (CI/CD).

A Pirâmide de Testes

Uma estratégia de testes equilibrada:

- ✓ **Base (Unitários):** Muitos, rápidos, isolados.
- ✓ **Meio (Integração):** Testam conexões entre componentes.
- ✓ **Topo (E2E):** Poucos, lentos, testam o fluxo real do usuário.

Testes Unitários

Focam na menor unidade de código possível (geralmente uma **função** ou **método**).

Características

Isolados: Não acessam banco de dados, rede ou sistema de arquivos.

Rápidos: Devem rodar em milissegundos.

Específicos: Se falharem, você sabe exatamente onde está o erro.

Integração e End-to-End

Testes de Integração

Verificam se diferentes módulos funcionam juntos.

Ex: Sua função de carregamento consegue ler do S3 corretamente?

Testes E2E (Ponta a Ponta)

Simulam o comportamento do usuário final.

Ex: O sistema recebe um CSV, processa e gera uma predição válida na saída?

Conheça o Pytest

O framework de testes mais popular do Python.

Por que Pytest?

- ✓ Sintaxe simples (apenas `assert`).
- ✓ Saída detalhada e colorida.
- ✓ Sistema poderoso de *fixtures*.
- ✓ Enorme ecossistema de plugins.



pytest

Descoberta Automática

O Pytest encontra seus testes automaticamente se você seguir as convenções de nome:

Arquivos

Devem começar ou terminar com `test_` ou `_test`.

Ex: `test_processamento.py`

Funções

Devem começar com `test_`.

Ex: `def test_media():`

Classes

Devem começar com `Test`.

Ex: `class TestModelo:`

Seu Primeiro Teste

Imagine que temos uma função simples em `math_utils.py`:

```
def quadrado(x): return x * x
```

O teste em `test_math.py` seria:

```
from math_utils import quadrado def test_quadrado_retorna_valor_correto(): resultado = quadrado(5) asser
```

A Mágica do `assert`

Em outros frameworks (como unittest), você precisa decorar dezenas de métodos: `assertEqual`, `assertTrue`, `assertListEqual`...

No **Pytest**, você só usa `assert`.

```
assert x == y assert lista_a == lista_b assert x in lista assert modelo.is_fitted is True
```

Se falhar, o Pytest mostra exatamente *por que* falhou (introspecção).

Rodando os Testes

No terminal, basta digitar:

```
$ pytest ===== test session starts ===== collected 1 item test_math.py . [100%
```

O . verde significa sucesso. Um F vermelho significaria falha.

Padrão AAA (Arrange-Act-Assert)

Estruture seus testes em três blocos claros:

1. Arrange

Prepare os dados e o ambiente.

Ex: Crie o DataFrame de entrada.

2. Act

Execute a função que está sendo testada.

Ex: Chame `processar_dados(df)`.

3. Assert

Verifique se o resultado é o esperado.

Ex: `assert output.shape == (10, 2)`.

O Problema do Ponto Flutuante

Computadores têm dificuldade com decimais precisos.

```
# Isso falha em Python! assert 0.1 + 0.2 == 0.3 # Resultado real: 0.30000000000000004
```

Solução: `py

Compare com uma

```
from pytest imp
```

Fixtures: Dados de Teste Reutilizáveis

Em vez de criar o mesmo DataFrame em todo teste (violando o princípio DRY), usamos **Fixtures**.

Fixtures são funções decoradas com `@pytest.fixture` que preparam dados antes do teste rodar e limpam depois.

Exemplo de Fixture

```
import pytest import pandas as pd @pytest.fixture def dados_exemplo(): # Arrange return pd.DataFrame({"
```

Parametrização

E se quisermos testar a mesma função com 10 entradas diferentes? Copiar e colar o teste 10 vezes?

Não! Usamos `@pytest.mark.parametrize`.

Isso executa a mesma função de teste várias vezes com argumentos variados.

Código Parametrizado

```
@pytest.mark.parametrize("entrada,esperado", [ (2, 4), (3, 9), (4, 16), (-2, 4), ]) def test_quadrado_v:
```

O Pytest tratará isso como **4 testes distintos** na contagem final.

Testando Erros (`pytest.raises`)

Um bom teste também verifica se o código **falha corretamente** quando deve falhar.

```
def test_divisao_por_zero(): with pytest.raises(ZeroDivisionError): divisao(10, 0)
```

Se `divisao(10, 0)` **não** levantar o erro, o teste falha!

Cobertura de Código (Coverage)

A métrica que diz qual porcentagem do seu código foi executada durante os testes.

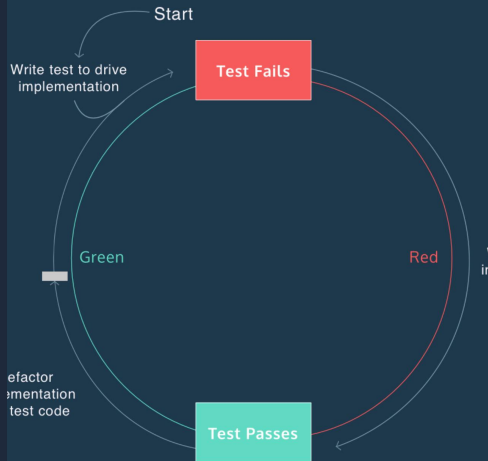
Ferramenta: `pytest-cov`

```
$ pytest --cov=meu_projeto
```

Cuidado: 100% de cobertura não garante ausência de bugs. Significa apenas que todas as linhas rodaram, não que a lógica está correta.

TDD: Desenvolvimento Orientado a Testes

1. ✓ **Red:** Escreva um teste que falha (antes do código existir).
2. ✓ **Green:** Escreva o código mínimo para o teste passar.
3. ✓ **Refactor:** Melhore o código sem medo (o teste garante).





Parte 2: Testando Dados

O código pode estar certo, mas e os dados?

Falhas Silenciosas

Em Data Science, erros de dados frequentemente não causam exceções (crashes). Eles causam **modelos ruins**.

- ✓ Uma coluna de idade com valores negativos.
- ✓ IDs duplicados onde deveriam ser únicos.
- ✓ Aumento súbito de valores nulos (Nulls).

Precisamos validar o **esquema** e as **propriedades estatísticas** dos dados.

Pandera: Validação de Pandas

Uma biblioteca leve e poderosa para definir esquemas de DataFrames.

Permite definir regras como:

- ✓ Tipos de dados (`int`, `float`, `string`).
- ✓ Intervalos (`Check.ge(0)`).
- ✓ Valores permitidos (`Check.isin(["A", "B"])`).

Se os dados não obedecerem, o Pandera lança um erro explicativo.

Exemplo Pandera

```
import pandera as pa esquema = pa.DataFrameSchema({ "idade": pa.Column(pa.Int, checks=pa.Check.ge(0)),
```

Great Expectations (GX)

Ferramenta mais robusta, focada em engenharia de dados.

Gera **Data Docs**: Documentação HTML automática sobre a qualidade dos seus dados.

Ideal para validar dados em produção (ETL pipelines).

Principais Expectativas

`expect_column_values_to_not_be_null`

`expect_column_mean_to_be_between`

Mocking (Simulação)

Testes unitários devem ser isolados. Mas e se seu código chama uma API externa ou Banco de Dados?

Usamos **Mocks**: Objetos falsos que simulam o comportamento de sistemas reais.

Permite testar "o que acontece se a API cair" sem derrubar a API de verdade.

Quando Mockar?



APIs Externas

Evite custos, limites de taxa e dependência de internet.



Banco de Dados

Evite configurar bancos complexos para testes simples.



Processos Lentos

Treinamento de modelos pesados (mock o método `.fit()`).

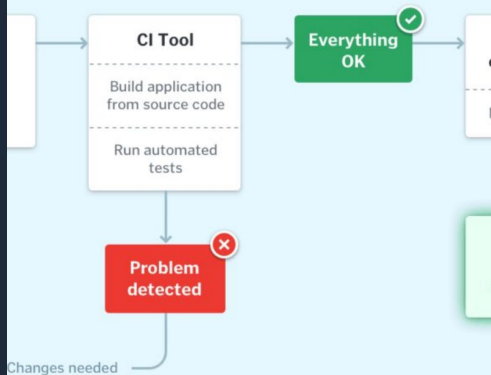
Integração Contínua (CI)

Testes só são úteis se forem rodados.

Serviços de CI (GitHub Actions, GitLab CI) rodam seus testes automaticamente toda vez que você empurra (push) código.

Se o teste falhar, o "Merge" é bloqueado.

Integration Workflow





Resumo

Escreva Testes

Use Pytest e foque em unitários.

Valide Dados

Use Panderas para garantir inputs.

Automatize

Rode tudo no CI.